

Jolly Take-Home Assessment

Junior Data Engineer

Estimated time: About 4 hours. There is no bonus for speed; we care much more about correctness, clear reasoning, and how you handle messy/ambiguous data than about how fast you finish.

Tools: Use whatever language/tools you're comfortable with (Python is a good default; SQL, Node.js, or similar are all fine). You may use libraries, docs, and the internet freely. Please don't use an AI assistant to write the whole solution for you. We want to see how you think, and we'll ask you to walk through your code in the follow-up conversation.

Submission: Reply with your code, the single merged transformed output file, and your short written answers (Part 2 and Part 3), either as a zip/folder or a link to a repo/gist.

Background

Jolly ingests employee/worker data from many different customer systems (EMRs, HR platforms, direct database connections) and transforms it into Jolly's internal accepted format (JAF) so the rest of the platform can work with it consistently. It's common for a single customer to hand us exports from more than one of their internal systems, and for the same person to show up in both.

For this exercise, imagine we're onboarding a new customer, MedNorth Health, and they've sent us exports from two different internal systems for the same group of staff:

- `sample_source_data.csv`: an export from their EMR (MedNorth EMR).
- `sample_source_data_2.json`: an export from their separate HR system (PeopleFlow HRIS), covering an overlapping set of people.

Both are small, realistic slices of what a first set of exports from a customer often looks like: inconsistent formatting, missing values, duplicates, at least one ambiguous field, and, since these come from two different systems tracking the same staff, some people who appear in both files. Your job is to turn both into a single, clean set of JAF records, one per person.

Part 1: Build the transformation (main task)

Write a script that reads both `sample_source_data.csv` and `sample_source_data_2.json` and outputs one merged file (CSV or JSON, your choice) where each row/record conforms to the JAF schema below: one JAF record per unique person, not one per source row.

Target JAF schema

Field	Type	Required	Notes
<code>source_system</code>	string	Yes	"MedNorth_EMR" or "PeopleFlow_HRIS". For a person found in both, pick one per your merge rule (see Part 2) and say which.
<code>external_id</code>	string	Yes	Record_ID (EMR) or employee_number (HRIS) from whichever source you treat as authoritative for that person
<code>first_name</code>	string	Yes	Parsed from Full_Name
<code>last_name</code>	string	Yes	Parsed from Full_Name
<code>date_of_birth</code>	date (ISO 8601, YYYY-MM-DD)	No	Null if missing or truly unresolvable; note any assumption made to parse ambiguous formats
<code>email</code>	string	No	Lowercase, trimmed; null if missing

Field	Type	Required	Notes
phone	string (E.164, e.g. +15551234567)	No	Null if missing
hire_date	date (ISO 8601, YYYY-MM-DD)	Yes	Normalized from Hire_Date
department	string	Yes	Mapped from Dept_Code using the table in Part 2; if unmapped, your call, document it
city	string	No	Parsed from City_State
state	string (2-letter)	No	Parsed from City_State
status	enum: "active" "inactive"	Yes	Normalized from Active_Flag (values include Y/N, 1/0, Active/Terminated, blank)

What your script needs to handle: MedNorth EMR (CSV)

- Name splitting: Full_Name appears both as "Last, First" and "First Last"; normalize into first_name / last_name.
- Date parsing: DOB and Hire_Date show up in at least four different formats (e.g. MM/DD/YYYY, DD-MM-YYYY, YYYY.MM.DD, YYYY/MM/DD). Normalize all dates to ISO 8601. At least one date is genuinely ambiguous; decide how to handle it and say why in your write-up.
- Contact fields: normalize email (lowercase, trimmed) and phone (E.164 format). Leave a field null rather than guessing when the source value is missing.
- Location: City_State is a combined, inconsistently formatted field (commas, dashes, no separator); split it into city and state.
- Status: Active_Flag uses several different representations of the same two states; normalize to "active" / "inactive".
- Department mapping: map Dept_Code to a full department name (see Part 2). One code in the sample data isn't in the mapping table; decide what your pipeline should do with it, and explain your choice.
- Duplicates: at least one pair of rows represents the same person submitted twice. Decide how your pipeline should handle this and document the logic.

What your script needs to handle: PeopleFlow HRIS (JSON)

- Structure: this export is nested JSON (name, contact, and address are sub-objects), not flat rows, so you'll need to flatten it before it can map to JAF.
- A different status vocabulary: employment_status uses values like "Employed", "Terminated", "Leave of Absence", and even a bare "T"; decide how each maps to JAF's active/inactive, and call out any value that doesn't map cleanly (e.g. "Leave of Absence").
- A different department scheme: department is free text with abbreviations (e.g. "Emerg.", "Peds", "Admin") rather than the EMR's short codes; use the alias table in Part 2, and note that these ultimately need to resolve to the same department names as the EMR source.
- Missing values: at least one record is missing birth_date, and one is missing state; handle the same way you handle missing values from the other source.

Merging the two sources

- At least two people appear in both files (matched by email). Your output should contain one JAF record per person, not one per source row; don't emit duplicates just because someone was in both exports.

- Where the two sources disagree on a field for the same person, decide which source "wins" (or how you'd reconcile it) and document the rule you used. There's no single correct answer here; we want to see your reasoning.
- Don't silently drop rows. If a record can't be fully mapped or matched with confidence, it should still show up in your output (with nulls where appropriate) or in a clearly separate "rejected/needs review" list. Your choice, just make it visible and explain it.

Part 2: Source mapping and assumptions (short write-up)

In a few short paragraphs or bullet points, write up:

1. The source-to-JAF field mapping you used for each source (a simple table or config, e.g. JSON/YAML, is fine; it doesn't need to be part of the running code).
2. How you handled the ambiguous date, the unmapped department code, and the within-source duplicate record, and why.
3. How you matched people across the two source files and resolved any conflicting field values between them, and why you chose that rule.
4. If this were a real customer onboarding, what 2-3 questions would you want to ask MedNorth's team before finalizing the mapping?

MedNorth EMR department code reference

Dept_Code	Maps to
ER	Emergency Medicine
PED	Pediatrics
ADM	Administration
CARD	Cardiology

PeopleFlow HRIS department alias reference

PeopleFlow "department" value	Maps to
Cardiology	Cardiology
Emerg.	Emergency Medicine
Admin	Administration
Peds	Pediatrics

Part 3: Short answer (a few sentences each)

1. A customer's connector starts failing intermittently overnight with timeout errors, but works fine when you test it manually during the day. What would you check first, and why?
2. You're asked to build a scraper against a customer portal that has no public API and occasionally changes its page structure. What would your approach be to make it reasonably resilient, and how would you know when it breaks?
3. A customer's HR system sends a field called "status" that doesn't cleanly match Jolly's expected active/inactive states (e.g. it includes values like "On Leave" or "Pending Start"). How would you decide what to do with those values, and who would you loop in?

What we're evaluating

- Correctness: does the output actually conform to the schema, including edge cases?
- Judgment: how you reason about ambiguous or messy data, not just whether you picked the "right" answer.
- Code quality: is it readable and reasonably organized, not necessarily fancy?
- Communication: is your write-up clear enough that a teammate could pick up your mapping logic without asking you 10 follow-up questions?

Questions about the assessment? Just reply and ask. We'd rather you ask than guess. Good luck!